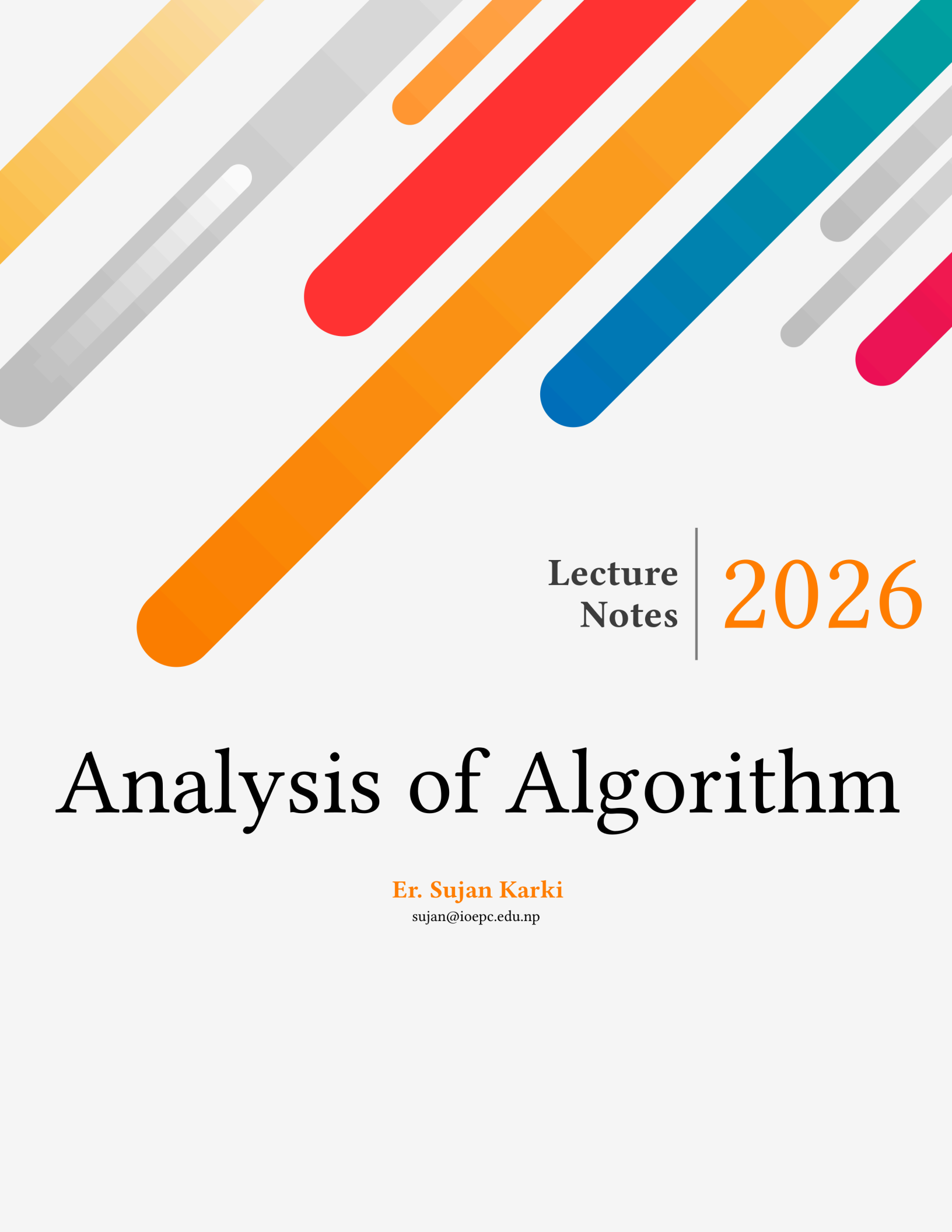




Lecture
Notes

2026

Analysis of Algorithm

Er. Sujan Karki

sujan@ioepc.edu.np

Chapter 2

Iterative and Numeric Algorithms

An **iterative algorithm** is an algorithm that repeatedly executes a set of instructions until a specific condition is met. Instead of calling itself (like recursion), it uses looping constructs such as:

- for loops
- while loops
- do-while loops

The idea is to repeat a process step-by-step and gradually move toward the final result.

2.1 Algorithm for GCD(a,b)

Algorithm 2.1.1

1. Start
2. If $a = 0$ AND $b = 0$, then return **Undefined GCD**
3. If $b = 0$, output $|a|$ and stop
4. If $a = 0$, output $|b|$ and stop
5. While $b \neq 0$ do:
 - remainder = $a \bmod b$
 - $a = b$
 - $b = \text{remainder}$
6. Output $|a|$
7. Stop

Example 2.1.1 (Trace table for Euclidean Algorithm.)

Step	a	b	$R = a \bmod b$ (remainder)
Initial	270	192	—
1	192	78	$270 \bmod 192 = 78$
2	78	36	$192 \bmod 78 = 36$
3	36	6	$78 \bmod 36 = 6$
4	6	0	$36 \bmod 6 = 0$

Loop stops when $b = 0$. So, GCD of number 270 and 192 is 6.

Euclidean Code

```

1 def gcd(a, b):
2     if a == 0 and b == 0:
3         print("GCD undefined")
4     elif a == 0:
5         print(f"{b} is GCD")
6     elif b == 0:
7         print(f"{a} is GCD")
8     else:
9         while b != 0:
10            r = a % b
11            a = b
12            b = r
13        print(f"{a} is GCD")

```

2.1.1 Time Complexity of Euclidean Algorithm

The Euclidean algorithm computes the greatest common divisor (GCD) of two numbers a and b as follows:

- Compute the remainder: $r = a \bmod b$
- Replace: $a \leftarrow b$ and $b \leftarrow r$
- Repeat until $b = 0$
- The GCD is the final value of a

At each step:

$$(a, b) \rightarrow (b, r) \quad \text{where } r = a \bmod b \text{ and } 0 \leq r < b$$

This ensures that the second number strictly decreases in every iteration, so the algorithm always terminates.

- The worst case occurs when the remainder decreases as slowly as possible.
- This happens when the inputs are consecutive Fibonacci numbers: F_n and F_{n-1} .

If $a = F_n$ and $b = F_{n-1}$, then:

$$F_n \bmod F_{n-1} = F_{n-2}$$

Next step:

$$(F_{n-1}, F_{n-2}) \rightarrow (F_{n-2}, F_{n-3}) \rightarrow \dots$$

Each iteration reduces the index of the Fibonacci number by 1. So, if $a = F_n$ and $b = F_{n-1}$, the number of steps is:

$$n - 1 \approx n$$

Example 2.1.2 (example)

The 36th Fibonacci number is 14, 930, 352 and the 35th Fibonacci number is 9, 227, 465.

$$14, 930, 352 \bmod 9, 227, 465 = 5, 702, 887$$

(This is the 34th Fibonacci number.)

Claim 2.1.1 Growth of Fibonacci Numbers

Since Fibonacci numbers grow exponentially:

$$F_n \approx \frac{\phi^n}{\sqrt{5}}$$

where ϕ (the golden ratio) is:

$$\phi = 1.61803398875$$

This means each Fibonacci number is roughly 1.618 times the previous number.

Fibonacci Approximation

$$F_k \approx \frac{\phi^k}{\sqrt{5}}, \quad \text{where } \phi = \frac{1 + \sqrt{5}}{2}$$

Worst Case Input

In the worst case, the Euclidean algorithm takes inputs as consecutive Fibonacci numbers:

$$b = F_{n-1}$$

Using the approximation:

$$b \approx \frac{\phi^{n-1}}{\sqrt{5}}$$

Take Logarithm (base ϕ)

$$\log_{\phi} b \approx \log_{\phi} \left(\frac{\phi^{n-1}}{\sqrt{5}} \right)$$

Apply Log Rules

$$\log_{\phi} b = (n - 1) - \log_{\phi}(\sqrt{5})$$

Solve for n

$$n \approx \log_{\phi} b + 1 + \log_{\phi}(\sqrt{5})$$

Ignore Constants

$$n \approx \log_{\phi} b$$

Change of Base (to base 2)

$$\log_{\phi} b = \frac{\log_2 b}{\log_2 \phi}$$

$$\log_2 \phi \approx 0.694$$

Final Approximation

$$n \approx \frac{\log_2 b}{0.694} \approx 1.44 \log_2 b$$

Big-O Notation

$$n = O(\log_2 b)$$

Final Result

Since $b = \min(a, b)$:

$$O(\log \min(a, b))$$

Reference: <https://www.techchorus.net/posts/euclids-algorithm-gcd/>

The smaller value acts as the limiting factor, since the algorithm repeatedly reduces the larger value by taking it modulo the smaller one. As a result, the speed of reduction—and therefore the total number of iterations—depends on the smaller value.

Time Complexity = $O(\log \min(a, b))$

2.1.2 Space Complexity

The algorithm uses a constant amount of memory, requiring only a single variable r . There is no use of recursion or additional data structures, so the space required does not grow with the size of the input.

$$\text{Space Complexity} = O(1)$$

2.2 Fibonacci Series

Definition 2.2.1

The Fibonacci series is a sequence of numbers where each number (after the first two) is the sum of the two preceding numbers:

$$F(0) = 0, \quad F(1) = 1$$

$$F(n) = F(n-1) + F(n-2), \quad n \geq 2$$

Example sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Algorithm 2.2.1

1. Start
2. If $n = 0$, return 0
3. If $n = 1$, return 1
4. Initialize $\text{prev} = 0$, $\text{curr} = 1$, $i = 2$
5. While $i \leq n$ do:
 - $\text{next} = \text{prev} + \text{curr}$
 - $\text{prev} = \text{curr}$
 - $\text{curr} = \text{next}$
 - $i = i + 1$
6. Return curr
7. Stop

Example 2.2.1 (Trace Table for Fibonacci Algorithm)

Step	prev	curr	next
Initial	0	1	—
1 ($i = 2$)	1	1	$0 + 1 = 1$
2 ($i = 3$)	1	2	$1 + 1 = 2$
3 ($i = 4$)	2	3	$1 + 2 = 3$
4 ($i = 5$)	3	5	$2 + 3 = 5$

Loop stops when $i > n$. For $n = 6$, Fibonacci number is 5.

Fibonacci Code

```

1 def fibonacci(n):
2     if n == 1:
```

```

3         return 0
4     if n == 2:
5         return 1
6     a, b = 0, 1
7     i = 3
8     while i <= n:
9         c = a + b
10        a = b
11        b = c
12        i += 1
13    return b

```

2.2.1 Time Complexity

Step-wise Analysis

```

1  def fibonacci(n):
2      if n == 1:                # O(1)
3          return 0              # O(1)
4      if n == 2:                # O(1)
5          return 1              # O(1)
6      a, b = 0, 1               # O(1)
7      i = 3                     # O(1)
8      while i <= n:             # (n - 2) iterations
9          c = a + b             # O(1)
10         a = b                  # O(1)
11         b = c                  # O(1)
12         i += 1                 # O(1)
13    return b                    # O(1)

```

Most statements in the algorithm are executed only once, regardless of the value of n . The main computation occurs inside the **while loop**. Each iteration of the loop performs a constant amount of work:

- one addition operation,
- a few assignments,
- and one increment operation.

Each of these operations takes constant time, i.e., $O(1)$. The loop runs from $i = 3$ to n , meaning it executes $(n - 2)$ times. Therefore, the total time taken by the loop is:

$$(n - 2) \times O(1).$$

Ignoring constants and lower-order terms, the overall time complexity is:

$$T(n) = O(n)$$

2.2.2 Space Complexity

Variables used: a, b, c, i

- No recursion is used
- No array or extra memory allocation

Since the number of variables is fixed and does not depend on input size,

$$\text{Space Complexity} = O(1)$$

2.3 Searching Algorithms

A **searching algorithm** is a step-by-step method used to **find a specific element (key)** in a collection of data such as an array, list, or file.

Searching algorithms are mainly classified into two categories:

- **Linear (Sequential) Search**
- **Binary Search**

In this chapter, we study both algorithms, their implementations, and complexity analysis.

2.4 Linear Search (Sequential Search)

Linear search checks **each element one by one** until the required element is found or the list ends. It works on both sorted and unsorted data.

Algorithm 2.4.1

1. Start
2. Read the search element from user.
3. Compare the search element with the first element in the list.
4. If match found → success. And display “Given element is found”.
5. Otherwise move to the next element.
6. Repeat until found or end of list.
7. If the last element on the list also doesn’t match then display “Element not Found”.
8. Stop

Code

```
1 def LinearSearch(A, n, key):
2     flag = 0
3     for i in range(n):
4         if A[i] == key:
5             flag = 1
6             break
7     if flag == 1:
8         print("Search successful")
9     else:
10        print("Search un-successful")
```

2.4.1 Time Complexity Analysis

There are three cases for Linear Search:

Best Case Complexity

The search key is found **at the first position** of the list. Only **1 comparison** is required.

$$T_{\text{best}}(n) = O(1)$$

Worst Case Complexity

The search key is found **at the last position** or **not present** in the list. So n **comparisons** are required.

$$T_{\text{worst}}(n) = O(n)$$

Average Case Complexity

Assume the key is equally likely to be at any position $1, 2, \dots, n$ (or not present). For simplicity, if present:

- Position 1 $\rightarrow$ 1 comparison
- Position 2 $\rightarrow$ 2 comparisons
- ...
- Position $n \rightarrow n$ comparisons

Total comparisons for all positions:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Average comparisons:

$$\frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \approx \frac{n}{2}$$

Hence,

$$T_{\text{avg}}(n) = O(n)$$

2.4.2 Space Complexity

The algorithm uses only a constant number of extra variables, such as i and flag . No additional arrays or dynamic memory are created during execution. Therefore, the required space remains constant regardless of the input size n .

$$\text{Space Complexity} = O(1)$$

2.4.3 Example

Example 2.4.1 (Search for key = 20 in the array: [12, 5, 10, 15, 31, 20, 25, 2, 40].

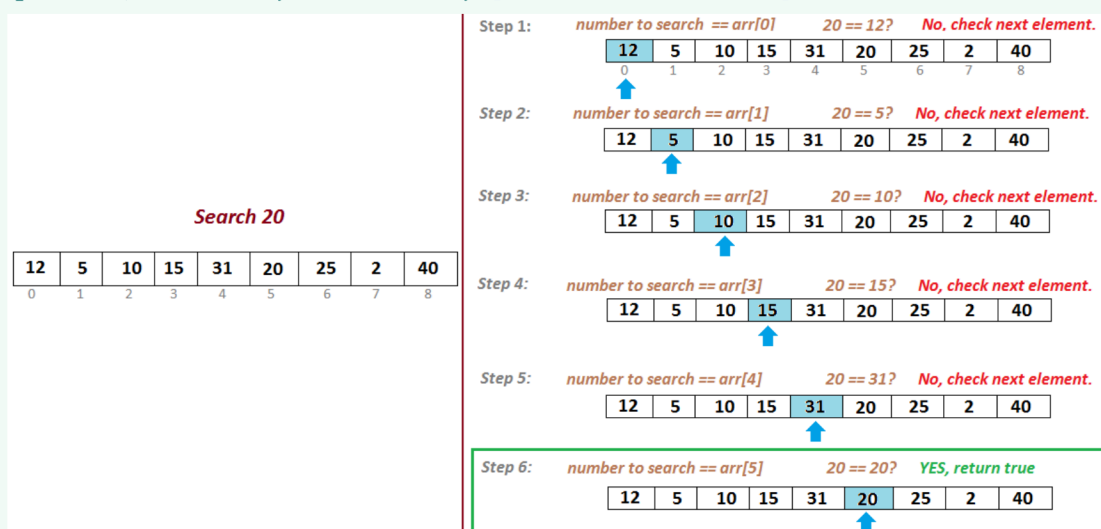


Figure 2.1: Tracing Linear Search

2.5 Bubble Sort

Definition 2.5.1: Bubble Sort

Bubble Sort is a straightforward, comparison-based sorting algorithm that is commonly used to introduce beginners to the concept of sorting in computer science. It works by repeatedly iterating through a list, comparing each pair of adjacent values, and swapping them if they are in the wrong order. This process is repeated until the list is completely sorted.

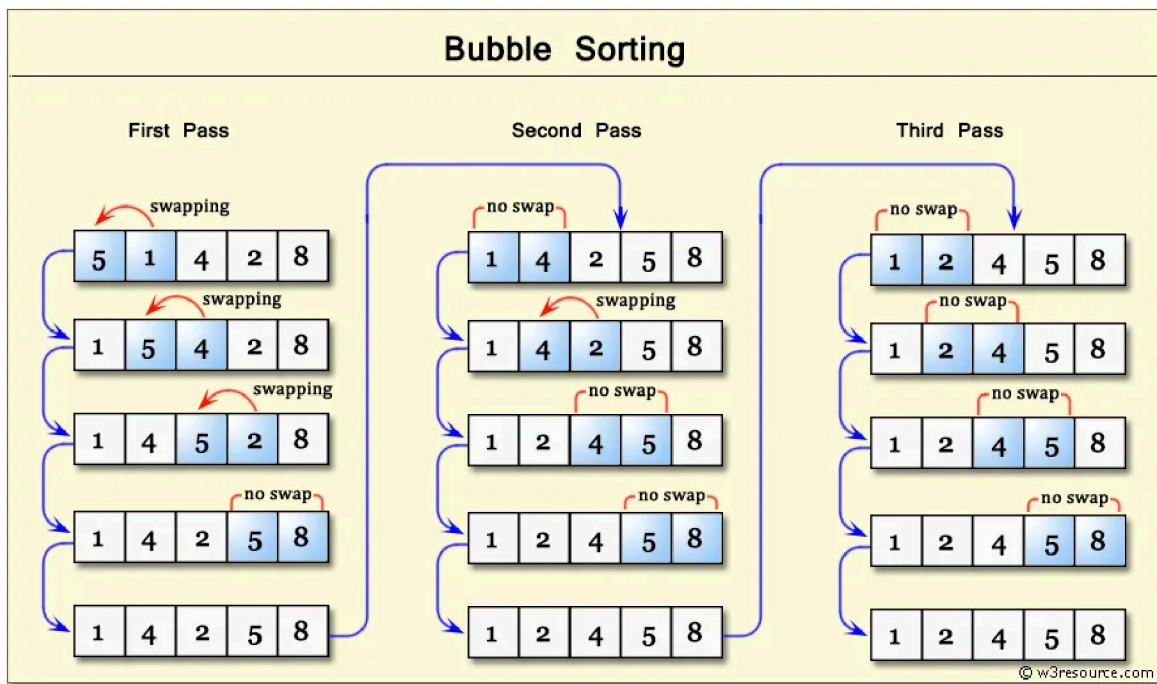


Figure 2.2: Tracing Bubble Sort

Note:-

Visualizer: <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

Algorithm 2.5.1

Input: Array A of size n

Output: Sorted array A in ascending order

1. Step 1: Start with the first element of the array.
2. Step 2: Repeat for each element in the array.
 - Compare each pair of adjacent elements.
 - If the current element is greater than the next element, i.e., if $A[j] > A[j + 1]$, then swap $A[j]$ and $A[j + 1]$.
 - Move to the next pair.
 - After each pass, the largest unsorted element moves to its correct position at the end.
 - Repeat passes until the entire array is sorted.
3. Step 3: Stop.

2.5.1 Pseudocode

Code

```

1  def bubble_sort(A):
2      n = len(A)
3      for i in range(n):
4          for j in range(0, n - i - 1):
5              if A[j] > A[j + 1]:
6                  A[j], A[j + 1] = A[j + 1], A[j]
7      return A

```

2.5.2 Time Complexity Analysis

Best Case:

The best case occurs when array is already sorted. The algorithm still makes $n - 1$ comparisons but "no swaps". With an early-exit (stop if no swap occurs), the best case becomes $O(n)$.

$$T_{\text{best}}(n) = O(n)$$

Worst Case:

The worst case occurs when array is sorted in descending order (reverse sorted). Every comparison leads to a swap.

Number of comparisons:

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

And the number of swaps is the same.

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

Total operations:

Total operations = Comparisons + Swaps

$$\begin{aligned}
 &= \frac{n(n - 1)}{2} + \frac{n(n - 1)}{2} \\
 &= n(n - 1)
 \end{aligned}$$

Therefore:

$$T_{\text{worst}}(n) = O(n^2)$$

Average Case:

Average Case:

The average case assumes the array elements are in random order. Bubble Sort always compares adjacent elements, regardless of input order.

So, the number of comparisons remains the same as worst case:

$$\begin{aligned}
 \text{Comparisons} &= (n - 1) + (n - 2) + (n - 3) + \cdots + 2 + 1 \\
 &= \frac{n(n - 1)}{2}
 \end{aligned}$$

On average, only half of the comparisons lead to swaps.

Total swaps (average case):

$$\text{Swaps} = \frac{1}{2} \times \frac{n(n - 1)}{2}$$

$$= \frac{n(n-1)}{4}$$

Total operations:

Total operations = Comparisons + Swaps

$$= \frac{n(n-1)}{2} + \frac{n(n-1)}{4}$$

Taking LCM:

$$= \frac{2n(n-1)}{4} + \frac{n(n-1)}{4}$$

$$= \frac{3n(n-1)}{4}$$

$$= \frac{3}{4}(n^2 - n)$$

$$T_{\text{avg}}(n) = O(n^2)$$

2.5.3 Space Complexity

It only uses a constant number of additional variables for indexing and swapping elements. Since no extra memory is allocated beyond those variables, the space requirement remains constant regardless of the size of the input array.

$$\text{Space Complexity} = O(1)$$

Note:-

$$O(f(n) + g(n)) = O(\max(f(n), g(n)))$$

Demo: <https://ds1-iiith.vlabs.ac.in/exp/bubble-sort/analysis/time-and-spce-complexity.html>

2.6 Selection Sort

Definition 2.6.1

The Selection sort algorithm is based on the idea of finding the minimum or maximum element in an unsorted array and then putting it in its correct position in a sorted array. The selection sort improves on the bubble sort by making only one exchange for every pass through the list.

Algorithm 2.6.1

1. Start
2. Input: Array A of size n (to be sorted in ascending order)
3. For $i = 0$ to $n - 2$ do: (boundary between sorted and unsorted)
 - Assume the current element is the minimum: $min = i$
 - For $j = i + 1$ to $n - 1$ do: (scan unsorted part)
 - If $A[j] < A[min]$ then $min = j$
 - If $min \neq i$, swap $A[i]$ and $A[min]$
4. Output sorted array A
5. Stop

2.6.1 Pseudocode

Code

```

1  def selection_sort(A):
2      n = len(A)
3      for i in range(n-1):
4          min_idx = i
5          for j in range(i+1, n):
6              if A[j] < A[min_idx]:
7                  min_idx = j
8          if min_idx != i:
9              A[i], A[min_idx] = A[min_idx], A[i]
10     return A

```

2.6.2 Time Complexity

Best Case

The best case occurs when the array is already sorted. Even in this situation, Selection Sort must scan the remaining unsorted portion in each iteration to find the minimum element. Therefore, it still performs all comparisons, and no swap is needed.

$$\text{Number of comparisons} = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

Hence,

$$T_{\text{best}}(n) = O(n^2)$$

Worst Case

The worst case occurs when the array is sorted in decreasing order. Even then, the number of comparisons remains the same and only swaps may differ. Every pass performs a swap (the minimum element is always at the far end). Number of passes = $n-1 \rightarrow$ Total swaps = $n-1$. So the worst case complexity still remains the same.

$$\text{Number of comparisons} = \frac{n(n-1)}{2}$$

$$T_{\text{worst}}(n) = O(n^2)$$

Average Case

In the average case, the elements can be in random order. Selection Sort does not take advantage of partial ordering. It still performs the same number of comparisons, while swaps vary because of position but remain at most $n-1$.

$$\text{Number of comparisons} = \frac{n(n-1)}{2}$$

$$T_{\text{avg}}(n) = O(n^2)$$

2.6.3 Space Complexity

Selection Sort is an in-place sorting algorithm. It does not use any additional data structures or dynamic memory that grows with input size. The sorting is performed within the original array itself using a constant number of variables.

Since only a fixed number of variables are used regardless of input size,

$$\text{Space Complexity} = O(1)$$

Note:-

Unlike Bubble Sort, Selection Sort minimizes the number of swaps (at most $n - 1$), but its comparison count is always $\frac{n(n-1)}{2}$, making it inefficient for large datasets. However, it performs well on small arrays and is easy to implement.

2.7 Insertion Sort

Definition 2.7.1

Insertion Sort is a simple comparison-based sorting algorithm that builds the final sorted array one element at a time. It works by repeatedly taking an element from the unsorted portion and inserting it into its correct position in the already sorted portion of the array. It works like how we sort playing cards in our hands.

Algorithm 2.7.1

1. Start
2. Input: Array A of size n (to be sorted in ascending order)
3. Start from the first element (already sorted) and consider the rest as unsorted.
4. For $i = 1$ to $n - 1$ do:
 - Set $temp = A[i]$ (the element to be inserted into the sorted part)
 - Set $j = i - 1$ (index of the last element in the sorted part)
 - While $j \geq 0$ and $A[j] > temp$ do:
 - Move $A[j]$ one position forward: $A[j + 1] = A[j]$
 - Decrease j by 1: $j = j - 1$
 - Insert $temp$ at the correct position: $A[j + 1] = temp$
5. Output sorted array A
6. Stop

2.7.1 Pseudocode

Code

```

1  def insertion_sort(A):
2      n = len(A)
3      for i in range(1, n):
4          temp = A[i]
5          j = i - 1
6          while j >= 0 and A[j] > temp:
7              A[j+1] = A[j]
8              j -= 1
9          A[j+1] = temp
10     return A

```

2.7.2 Time Complexity

Best Case

The best case occurs when the array is already sorted.

- The inner while loop never runs because $A[j] > temp$ is false for every element.
- Only the outer loop runs $n - 1$ times.
- so, Comparisons: $n - 1$
- Shifts (swaps): 0

$$T_{\text{best}}(n) = O(n)$$

Worst Case

The worst case occurs when the array is sorted in decreasing (reverse) order. In this case, every new element must be compared with all previously sorted elements and shifted to the beginning. i.e. Both outer and inner loops run for their maximum number of times.

$$\text{Number of comparisons} = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

$$\text{Total shifts:} = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

Total operations:

$$\begin{aligned} \text{Comparisons} + \text{Shifts} &= \frac{n(n - 1)}{2} + \frac{n(n - 1)}{2} \\ &= n(n - 1) \\ &= O(n^2) \end{aligned}$$

$$T_{\text{worst}}(n) = O(n^2)$$

Average Case

Assuming random order, Insertion Sort always run outer loop. So comparisons don't change much with input:

$$\text{Number of comparisons} = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

Not every comparison leads to a swap (unlike worst case). On average, if half of the comparisons will require a swap (statistical expectation).

$$\text{Total shifts:} = \frac{1}{2} * \frac{n(n - 1)}{2} = \frac{n(n - 1)}{4}$$

Total operations:

$$\begin{aligned} \text{Comparisons} + \text{Shifts} &= \frac{n(n - 1)}{2} + \frac{n(n - 1)}{4} \\ &= \frac{3n(n - 1)}{4} \\ &= O(n^2) \end{aligned}$$

$$T_{\text{avg}}(n) = O(n^2)$$

2.7.3 Space Complexity

Insertion Sort is an in-place algorithm. It uses only a constant number of extra variables: i , j , and $temp$.

$$\text{Space Complexity} = O(1)$$

Total space (including input array) = $O(n)$.

Note:-

Insertion Sort is efficient for small datasets or nearly sorted arrays (best case $O(n)$). It is often used as the final step in more advanced algorithms like Quicksort.

2.8 Number Theoretic Notations

Number theoretic notation refers to the standardized set of symbols, shorthand expressions, and notational conventions used to write and communicate ideas in number theory. Number theory deals with integers and their properties. Here are the essential notations used in number theoretic algorithms.

2.8.1 Basic Sets and Number Systems

- **Natural Numbers:** $\mathbb{N} = \{1, 2, 3, \dots\}$
- **Whole Numbers:** $\mathbb{N}_0 = \{0, 1, 2, 3, \dots\}$
- **Integers:** $\mathbb{Z} = \{0, \pm 1, \pm 2, \pm 3, \dots\}$
- **Even Integers:** $E = \{0, \pm 2, \pm 4, \pm 6, \dots\}$
- **Odd Integers:** $O = \{\pm 1, \pm 3, \pm 5, \pm 7, \dots\}$
- **Rational Numbers:** $\mathbb{Q} = \{p/q : p, q \in \mathbb{Z}, q \neq 0\}$
- **Real Numbers:** $\mathbb{R}$
- **Complex Numbers:** $\mathbb{C}$
- **Prime Numbers:** $\mathbb{P} = \{2, 3, 5, 7, 11, \dots\}$
- **Composite Numbers:** $C = \{4, 6, 8, 9, 10, \dots\}$

2.8.2 Divisibility

- **Notation:** $a \mid b$
- **Meaning:** a divides b exactly (no remainder).
- **Example:**

$$3 \mid 12 \quad \text{because } 12/3 = 4$$

$$5 \nmid 12 \quad \text{because } 12/5 = 2 \text{ remainder } 2$$

2.8.3 Modular Arithmetic / Congruence

- **Notation:** $a \equiv b \pmod{m}$
- **Meaning:** a and b leave the same remainder when divided by m .
- **Example:**

$$17 \equiv 5 \pmod{12}$$

$$23 \equiv 3 \pmod{10}$$

2.8.4 Greatest Common Divisor (GCD)

- **Notation:** $\gcd(a, b)$
- **Meaning:** The largest positive integer that divides both a and b .
- **Example:**

$$\gcd(48, 18) = 6$$

$$\gcd(a, 0) = a$$

2.8.5 Least Common Multiple (LCM)

- **Notation:** $\text{lcm}(a, b)$
- **Meaning:** The smallest positive integer divisible by both a and b .
- **Example:**

$$\text{lcm}(4, 6) = 12$$

2.8.6 Prime Numbers & Composite Numbers

- **Prime Number:** A prime number $p > 1$ has exactly two positive divisors: 1 and p itself.

Examples: 2, 3, 5, 7, 11, 13, ...

- **Composite Number:** A composite number $n > 1$ has more than two positive divisors.

Examples: 4, 6, 8, 9, 10, 12, ...

2.8.7 Coprime / Relatively Prime

- **Notation:** $\text{gcd}(a, b) = 1$
- **Meaning:** a and b have no common divisor other than 1.
- **Example:**

8 and 15 are coprime because $\text{gcd}(8, 15) = 1$

2.9 Euclid's Algorithm

Definition 2.9.1

In mathematics, Euclid's Algorithm is an efficient way for computing the greatest common divisor (GCD) of two numbers. It is named after the ancient Greek mathematician **Euclid**. Euclid's algorithm is a method to compute the greatest common divisor (GCD) of two integers by repeatedly replacing the larger number with the remainder of dividing it by the smaller, until the remainder becomes zero. The last non-zero remainder is the GCD.

2.9.1 Iterative Algorithm

Algorithm 2.9.1

1. Start
2. Input: Two non-negative integers a and b
3. While $b \neq 0$:
 - Set $r = a \bmod b$
 - Set $a = b$
 - Set $b = r$
4. Return a (now a is the GCD)
5. Stop

Iterative GCD

```

1  def gcd(a, b):
2      while b != 0:
3          r = a % b
4          a = b
5          b = r
6      return a

```

Complexity (Iterative)

- **Time Complexity:** $O(\log(\min(a, b)))$ (as studied in Chapter 2)
- **Space Complexity:** $O(1)$

$$T(n) = O(\log \min(a, b)),$$

$$S(n) = O(1)$$

2.9.2 Recursive Algorithm**Algorithm 2.9.2**

1. Start
2. Input: Two non-negative integers a and b
3. If $b = 0$, return a
4. Else return GCD($b, a \bmod b$)
5. Stop

Recursive GCD

```

1  def gcd(a, b):
2      if b == 0:
3          return a
4      else:
5          return gcd(b, a % b)

```

Complexity (Recursive)

- Each recursive call reduces the second argument from b to $a \bmod b$. So, number of recursive calls = $O(\log \min(a, b))$.
- **Time Complexity:** $O(\log \min(a, b))$.
- **Space Complexity:** $O(\log \min(a, b))$ because each call stores state on the call stack.

$$T(n) = O(\log \min(a, b)),$$

$$S(n) = O(\log \min(a, b))$$

2.9.3 Example

Example 2.9.1 (Find $\gcd(30, 21)$ by Euclid Algorithm.)

$$\text{GCD}(30, 21) \rightarrow \text{GCD}(21, 30 \bmod 21 = 9)$$

$$\text{GCD}(21, 9) \rightarrow \text{GCD}(9, 21 \bmod 9 = 3)$$

$$\text{GCD}(9, 3) \rightarrow \text{GCD}(3, 9 \bmod 3 = 0)$$

$$\text{GCD}(3, 0) \rightarrow 3$$

Hence, $\gcd(30, 21) = 3$.

Question 2.1

Find $\gcd(270, 192)$ using Euclid's Algorithm.

2.10 Extended Euclidean Algorithm

Definition 2.10.1

The Extended Euclidean Algorithm is an extension of the standard Euclidean Algorithm. While the Euclidean Algorithm finds the GCD of two integers a and b , the Extended Euclidean Algorithm also finds integers x and y such that:

$$ax + by = \gcd(a, b)$$

where x and y may be zero or negative. The main purpose is to express the GCD as a linear combination of a and b .

Algorithm 2.10.1

1. Start
2. Input: Integers a and b with $a \geq b \geq 0$
3. If $b = 0$, return $(a, 1, 0)$
4. Else:
 - Compute $(gcd, x_1, y_1) = \text{ExtendedGCD}(b, a \bmod b)$
 - Set $x = y_1$
 - Set $y = x_1 - \left\lfloor \frac{a}{b} \right\rfloor \cdot y_1$
 - Return (gcd, x, y)
5. Stop

Extended Euclidean Algorithm

```

1 def extended_gcd(a, b):
2     if b == 0:
3         return (a, 1, 0)      # gcd = a, x = 1, y = 0
4     else:
5         gcd, x1, y1 = extended_gcd(b, a % b)
6         x = y1

```

```

7      y = x1 - (a // b) * y1
8      return (gcd, x, y)

```

2.10.1 Time Complexity:

- In the worst case, the sequence of remainders follows Fibonacci numbers, which gives the maximum number of recursive calls.
- The number of recursive calls is therefore proportional to $O(\log(\min(a, b)))$.
- Each recursive call performs a constant amount of arithmetic (assignments, subtraction, multiplication) and are considered constant-time operations which are ignored in asymptotic analysis.

$$T(n) = O(\log \min(a, b))$$

2.10.2 Space Complexity:

- Because the algorithm is recursive, it uses the call stack.
- Depth of recursion = number of calls = $O(\log(\min(a, b)))$.

$$S(n) = O(\log \min(a, b))$$

2.10.3 Example

Example 2.10.1

Find numbers x and y such that $161x + 28y = \gcd(161, 28)$ using Extended Euclidean Algorithm.

$$\begin{aligned}
 \text{ExtendedGCD}(161, 28) &\rightarrow \text{ExtendedGCD}(28, 161 \bmod 28 = 21) \\
 \text{ExtendedGCD}(28, 21) &\rightarrow \text{ExtendedGCD}(21, 28 \bmod 21 = 7) \\
 \text{ExtendedGCD}(21, 7) &\rightarrow \text{ExtendedGCD}(7, 21 \bmod 7 = 0) \\
 \text{ExtendedGCD}(7, 0) &\rightarrow (7, 1, 0)
 \end{aligned}$$

Back substitution:

- For $(21, 7)$: $x = y_1 = 0, y = x_1 - \lfloor 21/7 \rfloor \cdot y_1 = 1 - 3 \cdot 0 = 1 \rightarrow (7, 0, 1)$
- For $(28, 21)$: $x = y_1 = 1, y = x_1 - \lfloor 28/21 \rfloor \cdot y_1 = 0 - 1 \cdot 1 = -1 \rightarrow (7, 1, -1)$
- For $(161, 28)$: $x = y_1 = -1, y = x_1 - \lfloor 161/28 \rfloor \cdot y_1 = 1 - 5 \cdot (-1) = 6 \rightarrow (7, -1, 6)$

Thus, $161 \cdot (-1) + 28 \cdot 6 = -161 + 168 = 7 = \gcd(161, 28)$.

2.10.4 Applications

- **Cryptography (RSA)** : Computes the modular inverse of the public exponent e modulo $\phi(n)$ to derive the private key d such that $e \cdot d \equiv 1 \pmod{\phi(n)}$.
- **Computer security** : Many cryptographic protocols (e.g., ECDSA, Diffie-Hellman) rely on modular inverses for key generation, signature verification, and secure key exchange — all computed efficiently via the Extended Euclidean Algorithm.

- **Competitive programming** : Solves linear Diophantine equations $ax + by = c$ and computes modular inverses for non-prime moduli where Fermat's theorem does not apply.

Question 2.2

1. Use the Extended Euclidean Algorithm to find $\gcd(198, 128)$ and integers x and y .
2. Why is the Extended Euclidean Algorithm used?

2.11 Chinese Remainder Theorem (CRT)

Definition 2.11.1

It is a theorem in number theory that helps us solve a system of simultaneous congruences with different moduli. It was first described in the 3rd century by the Chinese mathematician **Sun Tzu** in his book *Sunzi Suanjing*. The theorem states that if you know the remainders of an integer divided by several **pairwise coprime** moduli (divisors with no common factors), you can uniquely determine the number's remainder when divided by the product of those moduli.

2.11.1 Formal Statement

Suppose we have a system of congruences:

$$\begin{aligned} x &\equiv a_1 \pmod{m_1} \\ x &\equiv a_2 \pmod{m_2} \\ x &\equiv a_3 \pmod{m_3} \\ &\vdots \\ x &\equiv a_k \pmod{m_k} \end{aligned}$$

where $m_1, m_2, \dots, m_k$ are **pairwise coprime** (i.e., $\gcd(m_i, m_j) = 1$ for $i \neq j$). Then there exists a unique solution modulo

$$M = m_1 \times m_2 \times \dots \times m_k.$$

2.11.2 Method to Solve (for 3 congruences)

Given

$$\begin{aligned} x &\equiv a_1 \pmod{m_1} \\ x &\equiv a_2 \pmod{m_2} \\ x &\equiv a_3 \pmod{m_3} \end{aligned}$$

Step 1: Compute $M = m_1 \cdot m_2 \cdot m_3$.

Step 2: Compute $M_i = \frac{M}{m_i}$ for each i :

$$M_1 = \frac{M}{m_1}, \quad M_2 = \frac{M}{m_2}, \quad M_3 = \frac{M}{m_3}.$$

Step 3: Find the modular inverses y_i such that

$$M_i \cdot y_i \equiv 1 \pmod{m_i}.$$

Step 4: The solution is given by

$$x = (a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3) \pmod{M}.$$

Example 2.11.1 (Solving a System of Congruences using CRT)

Solve the system:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Step 1: $M = 3 \times 5 \times 7 = 105$.**Step 2:** Compute M_i :

$$M_1 = \frac{105}{3} = 35, \quad M_2 = \frac{105}{5} = 21, \quad M_3 = \frac{105}{7} = 15.$$

Step 3: Find the modular inverses y_i .We need values y_1, y_2, y_3 such that

$$M_i y_i \equiv 1 \pmod{m_i}.$$

- For $M_1 = 35$ and $m_1 = 3$:

Start with the original equation:

$$35y_1 \equiv 1 \pmod{3}.$$

Reduce 35 modulo 3:

$$35 \equiv 2 \pmod{3}.$$

So the equation becomes

$$2y_1 \equiv 1 \pmod{3}.$$

Try small values:

$$2 \times 1 = 2 \not\equiv 1 \pmod{3}$$

$$2 \times 2 = 4 \equiv 1 \pmod{3}$$

Therefore,

$$y_1 = 2.$$

- For $M_2 = 21$ and $m_2 = 5$:

Start with the original equation:

$$21y_2 \equiv 1 \pmod{5}.$$

Reduce 21 modulo 5:

$$21 \equiv 1 \pmod{5}.$$

So the equation becomes

$$1 \cdot y_2 \equiv 1 \pmod{5}.$$

Hence,

$$y_2 = 1.$$

- For $M_3 = 15$ and $m_3 = 7$:

Start with the original equation:

$$15y_3 \equiv 1 \pmod{7}.$$

Reduce 15 modulo 7:

$$15 \equiv 1 \pmod{7}.$$

So the equation becomes

$$1 \cdot y_3 \equiv 1 \pmod{7}.$$

Therefore,

$$y_3 = 1.$$

Step 4: Compute x :

$$\begin{aligned} x &= (a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3) \bmod M \\ &= (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \bmod 105 \\ &= (140 + 63 + 30) \bmod 105 \\ &= 233 \bmod 105 = 23. \end{aligned}$$

Thus the unique solution modulo 105 is $x \equiv 23 \pmod{105}$.

Note:-

The solution can be verified:

$$\begin{aligned} 23 \bmod 3 &= 2, \\ 23 \bmod 5 &= 3, \\ 23 \bmod 7 &= 2. \end{aligned}$$

Question 2.3: Assignment

Solve the following system of congruences:

$$\begin{cases} x \equiv 1 \pmod{2} \\ x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \end{cases}$$

2.12 Solving Modular Linear Equations

Definition 2.12.1

A modular linear equation has the form

$$ax \equiv b \pmod{m},$$

where a, b, m are integers, x is the unknown, and $m > 0$.

2.12.1 General Solution Method

Step 1: Compute $d = \gcd(a, m)$.

Step 2: Check if $d \mid b$.

- If $d \nmid b \rightarrow$ **no solution**.
- If $d \mid b \rightarrow$ **solutions exist**.

Condition	Number of Solutions
$d = 1$ (inverse exists)	1 solution
$d \mid b$ and $d > 1$	d solutions
$d \nmid b$	No solution

Step 3: If $d > 1$, the original equation can be simplified by dividing everything by d : Reduce the equation by dividing everything by d :

$$\frac{a}{d} \cdot x \equiv \frac{b}{d} \pmod{\frac{m}{d}}.$$

Now, $\gcd\left(\frac{a}{d}, \frac{m}{d}\right) = 1$, so the inverse exists.

Step 4: Find the modular inverse inv of $\frac{a}{d}$ modulo $\frac{m}{d}$:

That means: if modular inverse exists,

$$\frac{a}{d} \cdot inv \equiv 1 \pmod{\frac{m}{d}}.$$

Step 5: Multiply both sides of the reduced equation by the inverse inv .

Starting from

$$\frac{a}{d}x \equiv \frac{b}{d} \pmod{\frac{m}{d}},$$

multiply both sides by inv :

$$inv \left(\frac{a}{d}x \right) \equiv inv \left(\frac{b}{d} \right) \pmod{\frac{m}{d}}.$$

Rearranging,

$$\left(inv \cdot \frac{a}{d} \right) x \equiv \frac{b}{d} \cdot inv \pmod{\frac{m}{d}}.$$

Since

$$inv \cdot \frac{a}{d} \equiv 1 \pmod{\frac{m}{d}}$$

we obtain,

$$1 \cdot x \equiv \frac{b}{d} \cdot inv \pmod{\frac{m}{d}}.$$

Therefore,

$$x_0 \equiv \frac{b}{d} \cdot inv \pmod{\frac{m}{d}}.$$

Step 6: If $d > 1$, then there are d distinct solutions modulo m :

$$x = x_0 + t \cdot \frac{m}{d}, \quad t = 0, 1, \dots, d-1.$$

Example 2.12.1 (Solve the modular linear equation $14x \equiv 30 \pmod{100}$)

Step 1: Compute

$$d = \gcd(14, 100) = 2.$$

Step 2: Check whether $d \mid b$.

Since

$$2 \mid 30,$$

So, solutions exist.

Step 3: Divide the entire equation by $d = 2$:

$$\frac{14}{2}x \equiv \frac{30}{2} \pmod{\frac{100}{2}}$$

$$7x \equiv 15 \pmod{50}.$$

Now,

$$\gcd(7, 50) = 1,$$

So, the modular inverse of 7 modulo 50 exists.

Step 4: Find the modular inverse of 7 modulo 50.

We need a number inv such that

$$7 \cdot inv \equiv 1 \pmod{50}.$$

Trying values:

$$7 \times 43 = 301.$$

Since

$$301 = 6 \times 50 + 1,$$

we get

$$301 \equiv 1 \pmod{50}.$$

Therefore,

$$\text{inv} = 43.$$

Step 5: Multiply both sides of

$$7x \equiv 15 \pmod{50}$$

by the inverse 43:

$$43(7x) \equiv 43(15) \pmod{50}.$$

Rearranging,

$$(43 \times 7)x \equiv 645 \pmod{50}.$$

Since

$$43 \times 7 = 301 \equiv 1 \pmod{50},$$

we obtain

$$1 \cdot x \equiv 645 \pmod{50}.$$

Thus,

$$x \equiv 645 \pmod{50}.$$

Now reduce 645 modulo 50:

$$645 - 12 \times 50 = 645 - 600 = 45.$$

Hence,

$$x_0 \equiv 45 \pmod{50}.$$

Step 6: Since $d = 2$, there are 2 distinct solutions modulo 100:

$$x = x_0 + t \cdot \frac{m}{d}, \quad t = 0, 1.$$

Substituting the values:

$$x = 45 + t \cdot 50.$$

For $t = 0$:

$$x = 45.$$

For $t = 1$:

$$x = 95.$$

Therefore, the solutions are

$$x \equiv 45 \pmod{100}, \quad x \equiv 95 \pmod{100}.$$

2.13 Fermat's Little Theorem

Definition 2.13.1

Let p be a prime number and let a be any integer such that a is not divisible by p (i.e., $\gcd(a, p) = 1$). Then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Note:-

An equivalent formulation: For any integer a (not necessarily coprime to p),

$$a^p \equiv a \pmod{p}.$$

This version holds for all integers a (including multiples of p).

Example 2.13.1 (Take $p = 7$ and $a = 2$ (since $\gcd(2, 7) = 1$):

$$2^{7-1} = 2^6 = 64 \equiv 64 \bmod 7 = 1 \pmod{7}.$$

Indeed, $64 = 9 \times 7 + 1$.)

Example 2.13.2 (Take $p = 5$ and $a = 3$:

$$3^5 = 243 \equiv 243 \bmod 5 = 3 \pmod{5}.$$

Check: $243 = 48 \times 5 + 3$.)

2.13.1 Applications

Fermat's Little Theorem is widely used in:

- **Primality testing** (Fermat primality test)
- **Cryptography** (RSA encryption/decryption)
- **Computing modular inverses**: $a^{p-2} \equiv a^{-1} \pmod{p}$ when $\gcd(a, p) = 1$
- **Reducing large exponents modulo a prime**

2.13.2 Limitation of Fermat's Theorem

Note:-

The **converse** of Fermat's theorem is **false**. The converse would claim:

$$\text{If } a^{p-1} \equiv 1 \pmod{p} \text{ for all } a \text{ coprime to } p, \text{ then } p \text{ is prime.}$$

However, there exist **composite numbers** that also satisfy this property. These are called **Carmichael numbers**.

Example: $p = 561 = 3 \times 11 \times 17$ is composite, yet

$$a^{560} \equiv 1 \pmod{561}$$

holds for every integer a with $\gcd(a, 561) = 1$.

Because of these "false primes," the Fermat primality test is not reliable on its own. Stronger tests (like Miller–Rabin) are used to avoid Carmichael numbers.

Question 2.4: Practice Problem

Using Fermat's Little Theorem, compute $2^{100} \bmod 13$.

Step-by-Step Solution

1. **Apply Fermat's Little Theorem.** Since 13 is prime and $\gcd(2, 13) = 1$, we have

$$2^{12} \equiv 1 \pmod{13}.$$

2. **Rewrite the exponent.** Divide 100 by 12:

$$100 = 12 \times 8 + 4.$$

So we rewrite the exponent of 2^{100} using this:

$$2^{100} = 2^{12 \times 8 + 4}.$$

So altogether:

$$2^{100} = (2^{12})^8 \cdot 2^4.$$

3. **Substitute** $2^{12} = 1$ **into the expression.** We are not solving an equation here. We simply have the expression

$$(2^{12})^8 \cdot 2^4$$

and we know from Step 1 that 2^{12} and 1 are the same value in mod 13. So we swap 2^{12} with 1 inside the expression, just like substituting a known value:

$$\underbrace{(2^{12})^8}_{=1} \cdot 2^4 \equiv (1)^8 \cdot 2^4 \pmod{13}.$$

The $\equiv$ sign here simply means both sides are equal in mod 13. Nothing is being solved; we are just simplifying piece by piece.

4. **Simplify** $(1)^8$. Any power of 1 is still 1:

$$(1)^8 = 1.$$

So the expression becomes:

$$1 \cdot 2^4 \pmod{13}.$$

5. **Compute** 2^4 .

$$2^4 = 2 \times 2 \times 2 \times 2 = 16.$$

So we now have:

$$1 \times 16 = 16 \pmod{13}.$$

6. **Reduce** 16 **modulo** 13. Since $16 > 13$, subtract 13 once:

$$16 - 13 = 3.$$

Therefore:

$$16 \equiv 3 \pmod{13}.$$

7. **Conclusion.** The full chain of steps in one place:

$$2^{100} = 2^{12 \times 8 + 4} = (2^{12})^8 \cdot 2^4 \equiv (1)^8 \cdot 2^4 = 1 \cdot 16 = 16 \equiv 3 \pmod{13}.$$

Therefore,

$$\boxed{2^{100} \equiv 3 \pmod{13}}.$$

2.14 Primality Test: Miller-Rabin Randomized Primality Test

Definition 2.14.1: Miller-Rabin Primality Test

Primality testing is the process of determining whether a given number n is prime or composite. The Miller-Rabin test is a probabilistic algorithm used to check whether a number is prime. Randomized tests are faster and practical for large numbers, such as those used in cryptography.

Algorithm 2.14.1

MillerRabin(n, k):

if $n \leq 3$:

if $n = 2$ **or** $n = 3$: **return** True

else: **return** False

if n is even: **return** False

▷ Step 1: Find an odd number d such that $n - 1 = 2^s \cdot d$

To solve this, assume $d = n - 1$ and $s = 0$:

while $d \bmod 2 = 0$:

$d \leftarrow d // 2$

$s \leftarrow s + 1$

Finally we obtain values of s and d that satisfy $n - 1 = 2^s \cdot d$.

▷ Step 2: Perform k iterations, where k is the number of times we pick a random base and run the test.

for $i = 1$ to k :

$a \leftarrow$ choose a random number in range $[2, n - 2]$

$x \leftarrow a^d \bmod n$

if $x = 1$ **or** $x = n - 1$:

continue to next iteration

for $r = 1$ to $s - 1$:

$x \leftarrow x^2 \bmod n$

if $x = n - 1$: **break**

else: **return** False ▷ n is composite

return True ▷ n is probably prime

Miller-Rabin Primality Test

```

1  def miller_rabin(n, k):
2      # Handle small cases
3      if n <= 3:
4          return n == 2 or n == 3
5      if n % 2 == 0:
6          return False
7
8      # Step 1: Write n-1 = 2^s * d with d odd
9      d = n - 1
10     s = 0
11     while d % 2 == 0:
12         d //= 2
13         s += 1
14
15     # Step 2: Perform k iterations
16     for _ in range(k):
17         a = random.randint(2, n-2)
18         x = pow(a, d, n) # a^d mod n
19         if x == 1 or x == n-1:
20             continue
21         for _ in range(s-1):
22             x = pow(x, 2, n)
23             if x == n-1:

```

```

24         break
25     else:
26         return False    # n is composite
27     return True         # n is probably prime

```

2.14.1 Choosing k

Number Size	Recommended k
Small numbers (e.g., $n < 100$)	1 – 3
Medium numbers ($100 \leq n \leq 1000$)	5 – 10
Large cryptographic numbers	40 or more

2.14.2 Example

Example 2.14.1 (Testing $n = 13$ with $k = 2$)

Input: $n = 13$, $k = 2$. The initial conditions ($n \leq 3$ and n even) are both false, so we proceed.

Step 1: Find s and d such that $n - 1 = 2^s \cdot d$ with d odd.

$$n - 1 = 12 = 2^2 \times 3 \implies s = 2, \quad d = 3.$$

Step 2: Perform $k = 2$ iterations.

Iteration 1:

- Pick a random $a \in [2, n - 2] = [2, 11]$. Suppose $a = 4$.
- Compute $x = a^d \bmod n$:

$$x = 4^3 \bmod 13 = 64 \bmod 13 = 12.$$

- Since $x = 12 = n - 1$, we **continue** to the next iteration.

Iteration 2:

- Pick a random $a \in [2, 11]$. Suppose $a = 5$.
- Compute $x = a^d \bmod n$:

$$x = 5^3 \bmod 13 = 125 \bmod 13 = 8.$$

- Since $x = 8$ is neither 1 nor 12, enter the inner loop for $r = 1$ to $s - 1 = 1$:
 - $r = 1$: compute $x = x^2 \bmod n$:

$$x = 8^2 \bmod 13 = 64 \bmod 13 = 12.$$

Since $x = 12 = n - 1$, **break** out of the inner loop.

- Because we broke out with $x = n - 1$, we do **not** return False. The iteration passes.

Conclusion: Both iterations passed, so 13 is **probably prime**.

Note:-

The Miller-Rabin test is probabilistic: the probability that a composite number passes k iterations is at most 4^{-k} . For cryptographic applications, choosing $k = 40$ or more makes the error probability negligibly small.

Question 2.5

Using the Miller-Rabin Primality Test with $k = 2$, test whether $n = 17$ is prime. Use $a = 3$ for the first iteration and $a = 5$ for the second.

Question 2.6: Assignment

1. Write the algorithm for GCD and Fibonacci Number and analyze their time and space complexity.
2. Write the algorithm for Sequential Search and analyze its time and space complexity.
3. Write the algorithm for Bubble Sort, Selection Sort, and Insertion Sort. Analyze the time and space complexity of each.
4. Define Number Theoretic notations. Find $\gcd(270, 192)$ using the Euclidean Algorithm.
5. Use the Extended Euclidean Algorithm to find $\gcd(198, 128)$ and integers x and y such that

$$198x + 128y = \gcd(198, 128).$$

6. Why is the Extended Euclidean Algorithm used? Explain with an example.
7. State the Chinese Remainder Theorem. Solve the following system of congruences:

$$x \equiv 2 \pmod{3},$$

$$x \equiv 3 \pmod{5}.$$

8. Solve the modular linear equation

$$15x \equiv 35 \pmod{50}.$$

9. State Fermat's Little Theorem. Use it to compute:

$$3^{100} \bmod 7.$$

10. Explain the Miller-Rabin Randomized Primality Test. Test whether $n = 15$ is composite using the Miller-Rabin test with $k = 3$ iterations.
11. Test whether $n = 21$ is prime using the Miller-Rabin test with $k = 2$ iterations.